SMART CPU SCHEDULER WITH DYNAMIC PRIORITY ADJUSTMENT

OPERATING SYSTEMS PROJECT DOCUMENTATION

Course: Operating Systems

Team: G-2
Team Members and Roll Numbers:
1. Ratna priya - 2520030156
2. Teja Sri - 2520030132
3. Chandra Hasini - 2520030086

Department of Electronics and Communication Engineering
KLH University – Bachupally Campus
Academic Year: 2026–2027

Documentation Structure

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Existing System and Proposed System
- 6. Technologies Used
- 7. Methodology and Algorithm
- 8. Implementation and Source Code
- 9. Integration of OS Course Outcomes (CO1–CO6)
- 10. Results and Output
- 11. Performance Comparison
- 12. Testing, Limitations, Conclusion and Future Scope
- Appendix A: Build and Execution Instructions
- Appendix B: Important Formulas

Note: The documentation is based on the problem statement and the source files in the supplied smart_cpu_scheduler_hackathon.zip. Team member names/roll numbers are left blank because the supplied details were not sufficiently clear to enter reliably. Insert actual terminal screenshots and final measured scheduler outputs after running the program on your system.

1. Abstract

CPU scheduling is a core operating-system function that decides which ready process receives CPU time. Traditional priority scheduling can cause starvation when processes with lower priority are repeatedly postponed by higher-priority processes. This project implements a Smart CPU Scheduler with Dynamic Priority Adjustment using priority aging. The scheduler accepts each process's arrival time, burst time and initial priority, then simulates preemptive priority scheduling in one-time-unit steps. A process that waits for CPU service receives a better effective priority at fixed aging intervals. The program displays a Gantt-style execution sequence and calculates completion time, turnaround time, waiting time, response time, and their averages.

The project also contains a baseline scheduler without aging and separate C demonstrations aligned with all six Operating Systems course outcomes: OS services and system calls (CO1), process creation and control (CO2), inter-process communication (CO3), virtual memory (CO4), file-system I/O (CO5), and concurrency and synchronization (CO6). The main objective is to study scheduling fairness and connect the CPU scheduling problem with practical Linux system-programming concepts.

2. Introduction

2.1 Operating System and CPU Scheduling

An operating system manages the computer's hardware and provides services to applications. It manages processes, memory, files, devices and access to the CPU. Since multiple processes may be ready to execute at the same time, the scheduler chooses which process should run next.

2.2 Priority Scheduling

In priority scheduling, every process is assigned a priority. In this implementation, a smaller numeric value represents a higher priority: priority 0 is the highest, and priority 10 is the lowest initial priority accepted by the program. In preemptive priority scheduling, the scheduler re-evaluates the ready processes after each simulated time unit. If another ready process has a better effective priority, it can be selected for the next unit.

2.3 Starvation and Aging

Starvation is a situation in which a ready process waits for an excessively long time because other processes continually receive preference. Aging is a technique used to reduce starvation by gradually improving the effective priority of waiting processes. This project uses an aging interval of three simulated time units.

3. Problem Statement

In a computer system, multiple processes compete for CPU time, and the operating system must decide which process should execute at a particular moment. CPU scheduling plays an important role in improving system performance, reducing waiting time, and ensuring efficient utilization of CPU resources. One commonly used technique is priority scheduling, in which each process is assigned a priority and the CPU selects the process with the highest priority.

Traditional priority scheduling has a major limitation: starvation. A low-priority process may wait for a long time because higher-priority processes continuously receive preference. This can cause unfair resource allocation, increased waiting time and poor service for lower-priority processes. In preemptive priority scheduling, a newly arrived high-priority process may interrupt a currently running lower-priority process, making the lower-priority process wait longer.

To address this limitation, the project proposes a Smart CPU Scheduler with Dynamic Priority Adjustment using priority aging. The effective priority of a process is recalculated according to its waiting time. As a process waits, its effective priority improves, allowing it to compete more fairly for CPU execution. The proposed system is implemented in C and simulates preemptive priority scheduling. It accepts process ID (assigned by the program), arrival time, burst time and initial priority; simulates execution; produces a Gantt-style chart; and calculates completion time, turnaround time, waiting time, response time and average performance metrics. A baseline scheduler without aging is included so that both approaches can be tested with the same input. The project aims to reduce starvation risk, improve fairness and demonstrate how priority aging influences scheduling.

4. Objectives

- To understand the role of the CPU scheduler in process management.
- To implement preemptive priority scheduling in the C programming language.
- To apply dynamic priority aging so that waiting processes gradually receive better effective priority.
- To simulate process arrivals, CPU execution, preemption and completion.
- To display the execution order in a Gantt-style chart.
- To calculate completion time, turnaround time, waiting time and response time.
- To compare the aging scheduler with a baseline scheduler that does not use aging.
- To connect the main scheduling problem with OS services, process control, IPC, virtual memory, file I/O, and concurrency/synchronization concepts from CO1–CO6.
- To practise compiling, executing, testing and debugging C programs in a Linux environment.

5. Existing System and Proposed System

5.1 Existing System: Priority Scheduling Without Aging

The baseline program selects the ready process with the smallest original priority number. If two processes have the same priority, the implementation prefers the earlier arrival time; if those are also equal, it prefers the lower process ID. Because priority never improves, a low-priority process can repeatedly lose CPU time to newly arriving higher-priority processes.

5.2 Proposed System: Priority Scheduling With Aging

The proposed scheduler recalculates effective priority for every ready process at each simulated time step. The implemented aging interval is AGING_INTERVAL = 3. The effective priority decreases numerically as the process accumulates waiting time, but never goes below zero. Since a lower number means a higher priority, this change gives long-waiting processes a better chance to be selected.

Feature	Baseline scheduler	Proposed scheduler
Scheduling type	Preemptive priority simulation	Preemptive priority simulation
Priority used	Original priority	Dynamically calculated effective priority
Aging	Disabled	One priority-level improvement per 3 waiting time units
Starvation risk	Can be high for low-priority processes	Reduced, but depends on workload and priority rules
Output	Gantt sequence and CT/TAT/WT/RT metrics	Gantt sequence, aging messages and CT/TAT/WT/RT metrics
Input limits	1–50 processes; arrival ≥ 0; burst > 0; priority 0–10	Same

6. Technologies Used

Technology / Tool	Purpose
C (C11)	Implements the scheduler and OS concept demonstrations.
GCC compiler	Compiles the C source files into executable programs.
Ubuntu/Linux terminal	Runs POSIX system calls and Linux-oriented demonstrations.
POSIX APIs	Demonstrates process creation, pipes, signals, file descriptors, memory mapping and threads.
Makefile	Builds the main scheduler and CO1–CO6 demonstration programs.
Text output / result log	Displays the Gantt chart, metrics and saved results.

The supplied Makefile uses GCC with -Wall -Wextra -std=c11. The concurrency demonstration is compiled with -pthread.

7. Methodology and Algorithm

7.1 Input

The program asks for the number of processes (between 1 and 50). For each process, the user enters arrival time (AT), burst time (BT) and initial priority. Process IDs are assigned automatically as P1, P2, and so on. Arrival time must be non-negative, burst time must be positive, and initial priority must be in the range 0–10.

7.2 Effective Priority Formula

Let AT be arrival time, BT be burst time, R be remaining burst time, and t be the current simulated time.

$$\textbf{Executed time} = BT - R$$

$$\textbf{Waiting time so far} = \max(0, t - AT - \text{Executed time})$$

$$\textbf{Effective priority} = \max(0, \text{Initial priority} - \text{floor}(\text{Waiting time so far} / 3))$$

The formula follows the implementation in src/scheduler.c. Each complete group of three waiting time units improves the numeric priority by one. The effective priority is capped at zero. The code calculates waiting time so far as elapsed time since arrival minus time already executed, so time spent running is not counted as waiting.

7.3 Scheduling Algorithm

1. Read and validate the process count and each process's AT, BT and priority.
2. Initialize remaining burst time to BT, first-start time to −1 and completion time to zero.
3. Set the simulated clock to zero and completed-process count to zero.
4. At each time step, examine all processes that have arrived and still have remaining CPU time.
5. Calculate each ready process's effective priority using the aging formula.
6. Select the process with the smallest effective priority. Break ties by earlier arrival time, then lower process ID.
7. If no process is ready, record an idle time slot and advance the clock.
8. Otherwise, record the process's first start time if it has not started before, print its one-unit Gantt segment, reduce remaining burst time by one and advance the clock.
9. When remaining burst time becomes zero, record completion time and increase the completed-process count.
10. Repeat until all processes complete, then calculate and print per-process and average metrics.

7.4 Performance Metrics

Metric	Formula	Meaning
Completion Time (CT)	Time at which process finishes	Finish time of the process.
Turnaround Time (TAT)	CT − AT	Total time from arrival to completion.
Waiting Time (WT)	TAT − BT	Time spent waiting rather than executing.
Response Time (RT)	First CPU start time − AT	Time from arrival until first execution.
Average WT	Sum of all WT / number of processes	Mean waiting time for the workload.
Average TAT	Sum of all TAT / number of processes	Mean turnaround time for the workload.
Average RT	Sum of all RT / number of processes	Mean response time for the workload.

8. Implementation and Source Code

8.1 Main Scheduler

File: src/scheduler.c. This is the main proposed implementation. It defines a Process structure containing PID, arrival time, burst time, remaining time, initial priority, first start time and completion time. The effective_priority() function calculates the current effective priority. The main scheduling loop selects one ready process per time unit, which simulates preemptive execution at discrete time boundaries.

Important source-code section: aging formula

```
static int effective_priority(const Process *p, int time)
{
    int executed = p->burst - p->remaining;
    int waited = time - p->arrival - executed;

    if (waited < 0)
        waited = 0;

    int priority = p->priority - waited / AGING_INTERVAL;

    return priority < 0 ? 0 : priority;
}
```

8.2 Baseline Scheduler

File: src/baseline_scheduler.c. This program uses the same input format and metric formulas, but it always selects using the original priority. Running both programs with identical input provides a fairer comparison of the effect of aging.

8.3 Project Folder Structure

```
smart_cpu_scheduler_hackathon/
├── Makefile
├── src/
│   ├── scheduler.c
│   └── baseline_scheduler.c
├── modules/
│   ├── co1_os_services/co1_demo.c
```

```
│   ├── co2_process_control/co2_demo.c
│   ├── co3_ipc/
│   │   ├── scheduler_pipe.c
│   │   ├── fifo_demo.c
│   │   ├── signal_demo.c
│   │   ├── shared_memory_demo.c
│   │   └── unix_socket_demo.c
│   ├── co4_virtual_memory/memory_demo.c
│   ├── co5_file_system/scheduler_file_io.c
│   └── co6_concurrency/concurrency_demo.c
└── results/
    └── scheduler_results.txt
```

The Makefile builds the main scheduler and the CO1–CO6 demonstration programs. The baseline_scheduler.c file is present in the project source, but the supplied Makefile does not currently include a baseline_scheduler target; compile it separately using the command in Appendix A.

9. Integration of OS Course Outcomes (CO1–CO6)

The project uses the CPU scheduler as its central theme and includes separate demonstrations to connect it to the complete OS handout. These demonstrations support the concepts; they do not mean every feature is part of the main scheduler's core scheduling loop.

9.1 CO1: The OS as a Service Layer

Topics from the handout: user space and kernel space, system calls, shell role, command execution, Linux architecture and systems-programming fundamentals.

Project integration: modules/co1_os_services/co1_demo.c demonstrates write(), getpid(), read() and standard input/output. write() sends a message to standard output, getpid() reports the process ID, and read() accepts a short message from the terminal. These calls illustrate how a user-space program requests OS services through system-call interfaces.

9.2 CO2: Processes and Process Control

Topics from the handout: process abstraction and lifecycle, state transitions, creation, execution, synchronization, termination, scheduling concepts and process-management pitfalls.

Project integration: the main scheduler represents processes using a structure and models arrival, ready/waiting, running and completed states. modules/co2_process_control/co2_demo.c demonstrates fork() to create a child, execlp() to replace the child's program image, and waitpid() so the parent waits for child completion. The scheduler itself is a simulation; it does not change the Linux kernel's real CPU scheduling policy.

9.3 CO3: Inter-Process Communication (IPC)

Topics from the handout: anonymous pipes, named pipes/FIFOs, signals, signal handlers, POSIX signal programming, process groups, sessions and job control.

Project integration: modules/co3_ipc contains multiple IPC examples. scheduler_pipe.c sends a scheduler-task structure through an anonymous pipe between parent and child. fifo_demo.c uses a named FIFO for a scheduling message. signal_demo.c demonstrates SIGUSR1 as a notification. shared_memory_demo.c shares a data structure through mmap() with MAP_SHARED. unix_socket_demo.c demonstrates bidirectional communication using a local socket pair. Together, these examples show alternative mechanisms for transferring scheduler-related data or notifications between processes.

9.4 CO4: Memory Management and Virtual Memory

Topics from the handout: virtual memory, address translation, page tables, page faults, demand paging, process address-space layout, dynamic allocation, copy-on-write, memory errors and debugging.

Project integration: modules/co4_virtual_memory/memory_demo.c maps a 1 MiB region with mmap(), writes to one location per page and reports minor/major page-fault counters using getrusage(). It then forks a child to demonstrate copy-on-write: the child modifies its mapping while the parent checks that its own value remains unchanged. This demonstrates OS memory behaviour relevant to running processes.

9.5 CO5: File Systems and File I/O

Topics from the handout: Unix file abstraction, inodes, directory entries, VFS, file descriptors, open-file management, Linux file I/O system calls, buffered/unbuffered I/O, memory-mapped I/O and ext4 overview.

Project integration: modules/co5_file_system/scheduler_file_io.c runs an aging scheduler and saves the generated result text to results/scheduler_results.txt using open() and write(), then reopens and reads the file using read(). The supplied results/scheduler_results.txt records one run with an aging interval of three time units. File descriptors and system calls connect the scheduler's results to the Unix file abstraction.

9.6 CO6: Concurrency and Synchronization

Topics from the handout: threads and processes, POSIX threads, race conditions, shared data, mutexes, condition variables, semaphores, deadlocks and advanced synchronization.

Project integration: modules/co6_concurrency/concurrency_demo.c creates three worker threads, each processing four tasks. A condition variable and mutex coordinate when workers start; a semaphore limits simultaneous access to a resource to two workers; another mutex protects the shared completed_tasks counter. The final count is checked against the expected total of twelve tasks. This illustrates why shared state requires synchronization.

Course outcome	Relevant source file(s)	Concept demonstrated
CO1	modules/co1_os_services/co1_demo.c	System calls, process ID and terminal I/O
CO2	src/scheduler.c; src/baseline_scheduler.c; modules/co2_process_control/co2_demo.c	Scheduling simulation, fork/exec/wait
CO3	modules/co3_ipc/*.c	Pipe, FIFO, signal, shared memory and Unix socket
CO4	modules/co4_virtual_memory/memory_demo.c	mmap, page-fault counters and copy-on-write
CO5	modules/co5_file_system/scheduler_file_io.c	Persist and read scheduler output using file I/O
CO6	modules/co6_concurrency/concurrency_demo.c	Threads, mutex, condition variable and semaphore

10. Results and Output

10.1 Main Scheduler Output

When executed, the main program prints the scheduler title, aging interval, input prompts, a Gantt-style chart and a metrics table. It also prints an [AGING] message when a ready process's effective priority becomes numerically smaller than its previously recorded effective priority.

The Gantt chart displays one-unit execution segments. A segment such as P2(Pri=1,4-5) means process P2 ran from simulated time 4 to 5 with effective priority 1. An idle(t) segment means no process was ready at that time.

10.2 Metrics Table

The final table contains PID, AT, BT, original priority, CT, TAT, WT and RT. Average waiting time, average turnaround time and average response time are printed below the table. These values should be captured from a real run of the supplied program and inserted as a screenshot in the final submission.

10.3 Existing Saved File-I/O Result

The ZIP contains a saved CO5 demonstration result with aging interval 3. It records P1 (arrival 1, burst 2, priority 1) and P2 (arrival 2, burst 2, priority 2), with average turnaround time 2.50 and average waiting time 0.50. This is the recorded CO5 file-I/O example, not a universal result for all inputs.

```
Smart CPU Scheduler - CO5 File I/O Results
Aging interval: 3 time units

PID Arrival Burst   Priority       Completion     Turnaround     Waiting
P1  1       2       1              3              2              0
P2  2       2       2              5              3              1

Average turnaround time: 2.50
Average waiting time: 0.50
```

10.4 Screenshots to Insert

- Terminal showing successful compilation with make.
- Main aging scheduler input, Gantt chart, aging messages and metrics.
- Baseline scheduler output for exactly the same input.
- CO1 and CO2 demonstrations.
- At least one CO3 IPC demonstration.
- CO4 memory demonstration, including page-fault and copy-on-write messages.
- CO5 message showing results saved and read back from the file.
- CO6 output showing all workers finished and completed task count = 12.

11. Performance Comparison

For a meaningful comparison, enter exactly the same process count, arrival times, burst times and initial priorities into both executables. Record the average waiting time, average turnaround time and average response time from each run.

Metric	Without aging	With aging	Interpretation
Average waiting time	Fill from baseline output	Fill from scheduler output	Lower is generally better, but fairness should also be considered.
Average turnaround time	Fill from baseline output	Fill from scheduler output	Lower means processes finish sooner on average.
Average response time	Fill from baseline output	Fill from scheduler output	Lower means processes start sooner on average.
Starvation/fairness observation	Describe any process delayed by higher-priority arrivals	Describe whether aging improves its effective priority and service	Use the actual Gantt charts and per-process values.

Do not claim that aging always lowers every average metric. Aging is primarily intended to improve fairness and reduce starvation risk; it can change the execution order and may improve some metrics while worsening others. The comparison must be based on measured output.

12. Testing, Limitations, Conclusion and Future Scope

12.1 Test Cases

Test	Input condition	Expected behaviour
Valid input	1–50 processes; valid AT, BT and priority	Program accepts values and prints chart and metrics.
Invalid process count	0 or more than 50	Program reports invalid process count and exits.
Invalid arrival time	Negative arrival time	Program reports invalid input and exits.
Invalid burst time	Burst time is zero or negative	Program reports invalid input and exits.
Invalid priority	Priority outside 0–10	Program reports invalid input and exits.
Idle CPU	First process arrives after time zero	Gantt chart includes idle time until a process arrives.
Aging behaviour	A process waits across multiple aging intervals	Its effective priority improves numerically, down to a minimum of zero.
Baseline comparison	Same workload in both executables	Both produce per-process metrics that can be compared.
CO6 synchronization	Run concurrency_demo	All 12 tasks should complete and be reported.

12.2 Limitations

- The scheduler is a user-space simulation; it does not modify the Linux kernel scheduler or schedule actual OS processes.
- Execution is simulated in one-time-unit steps and does not model real context-switch overhead.
- The aging interval is fixed at three time units in the source code.
- Effective priority is capped at zero, so processes can eventually reach the same best priority; tie-breaking then affects execution order.
- The supplied Makefile does not build baseline_scheduler by default, so it must be compiled separately unless the Makefile is updated.
- The saved results file is a specific recorded example; new input produces different results.
- The supplied source code demonstrates the CO concepts through separate small programs rather than integrating every OS mechanism into the scheduler's main loop.

12.3 Conclusion

The Smart CPU Scheduler with Dynamic Priority Adjustment demonstrates how priority aging can address one of the major weaknesses of traditional priority scheduling: starvation. The program accepts process attributes, simulates preemptive execution, recalculates effective priorities, produces a Gantt-style chart and computes standard scheduling metrics. The baseline implementation provides a reference for comparison. Additional modules connect the project to CO1–CO6 through system calls, process control, IPC, virtual memory, file I/O, and thread synchronization. Overall, the project provides both a scheduling simulation and practical Linux systems-programming demonstrations.

12.4 Future Scope

- Make the aging interval configurable at runtime.
- Add a CSV export option for scheduling metrics and comparison tables.
- Generate a graphical Gantt chart instead of terminal-only output.
- Add automated tests with known expected scheduling results.
- Compare priority aging with FCFS, Round Robin and Shortest Job First using common workloads.
- Add configurable priority direction, tie-breaking policies and aging rules.
- Measure fairness with per-process maximum waiting time and starvation indicators.
- Improve the Makefile so it builds and cleans the baseline executable as well as all demonstrations.

Appendix A: Build and Execution Instructions

Run these commands from the extracted project directory on Ubuntu/Linux with GCC and make installed.

```
# Build the targets listed in the Makefile
make

# Run the proposed scheduler with aging
./scheduler

# Compile and run the baseline scheduler separately
gcc -Wall -Wextra -std=c11 src/baseline_scheduler.c -o baseline_scheduler
./baseline_scheduler

# Run the OS concept demonstrations
./modules/co1_os_services/co1_demo
./modules/co2_process_control/co2_demo
./modules/co3_ipc/scheduler_pipe
./modules/co3_ipc/fifo_demo
./modules/co3_ipc/signal_demo
./modules/co3_ipc/shared_memory_demo
./modules/co3_ipc/unix_socket_demo
./modules/co4_virtual_memory/memory_demo
./modules/co5_file_system/scheduler_file_io
./modules/co6_concurrency/concurrency_demo

# Remove the binaries built by the Makefile
make clean
```

For the main scheduler, enter the process count and then one line per process containing: arrival_time burst_time priority. Example format only: 0 5 3. Use the same workload in both scheduler programs for comparison.

Appendix B: Important Formulas

Term	Formula
Executed time	BT − remaining burst time
Waiting so far	max(0, current time − arrival time − executed time)
Effective priority	max(0, initial priority − floor(waiting so far / 3))
Turnaround time	CT − AT
Waiting time	TAT − BT
Response time	First start time − AT
Average metric	Sum of process metric values / number of processes

Final submission checklist: replace the cover-page blanks with the correct team names and roll numbers; add the university-required logo if required; insert actual screenshots; and fill in the performance comparison from a matched run of both executables.